

Polymorfism

Det fina ordet polymorfism innebär i praktiken något som kan ses som *överlagring av funktioner och metoder*. Med överlagring avser vi dock inte här i samband med arv, utan på bas av de argument som en funktion accepterar. Vi har redan sett en form av polymorfism i samband med multipla konstruktörer till en klass (se [avsnittet Multipla konstruktörer i Kapitel 14](#)). Multipla konstruktörer ger oss möjligheter att skapa olika typer av konstruktörer för att göra det enkelt att instantiera klassen. Ett exempel på en klass med multipla konstruktörer är:

```
class Line {
public:
    // multipla konstruktörer
    Line ();
    Line (const Coordinate & Source, const Coordinate & Target);
    Line (float X1, float Y1, float X2, float Y2);

    ...
};
```

Vi kan skapa ett `Line`-objekt på tre olika sätt. Beroende på vilken form av data vi har då objektet skall skapas kan vi troligtvis använda det direkt utan att behöva t.ex. konvertera rena `float`-värden till `Coordinate` eller vice versa. Kompilatorn använder sig av information ur sammanhanget och om de använda parametrarna för att välja vilken konstruktör som används. Om ingen konstruktör finns direkt för de givna parametrarna kollar kompilatorn om det finns något direkt sätt att konvertera de givna parametrarna till någonting som skulle duga. Vi kunde t.ex. instantiera en `Coordinate` med fyra `int`:

```
int X1, X2, Y1, Y2;
Coordinate C ( X1, X2, Y1, Y2 );
```

Funktions- och metodpolymorfism

Förutom endast konstruktörer kan polymorfism användas för normala funktioner och metoder. Detta gör det enkelt att skapa metoder som accepterar argument av olika datatyp. Om vi tänker oss en funktion som skriver ut data av olika typ på skärmen måste vi utan polymorfism använda metoder med olika namn:

```
void print_int (int Value) { ... }
void print_string (const string & Value) { ... }
void print_Person (const Person & Value) { ... }
```

Det blir ganska fult att göra utskriften på detta sätt. Det är svårt att minnas vilka metoder som kan användas och det är svårare att skriva generiska program. Istället kan man använda sig av polymorfism om skriva funktionerna så här:

```
void print (int Value) { ... }
void print (const string & Value) { ... }
void print (const Person & Value) { ... }
```

En hel del snyggare än det första. Nu behöver man inte längre fundera på vad funktionen heter, eftersom den alltid heter `print()`.

Notera att det inte är möjligt att ha polymorfism med avseende på *returvärde*. Följande exempel där vi har två metoder, som inte tar några argument, för att skapa objekt av klassen `Circle` och klassen `Rectangle` är inte möjligt:

```
Circle * C = ShapeFactory.create ();  
Shape * S = ShapeFactory.create ();
```

Kompilatorn vet inte vilken metod som skall anropas i det senare fallet. Skall man använda metoden som skapar en `Circle` eller den som skapar en `Rectangle`, eftersom båda är möjliga.

[Förutgående](#)[Friends](#)[Hem](#)[Upp](#)[Nästa](#)[Standardvärden för funktioner
och metoder](#)